

Next 10 Geometry + State Estimation Problems

Simple C++17 interview approaches with the minimum math needed to explain each solution.

Problem	Method	Time
Closest Point on a Line Segment	Projection + clamp	$O(1)$
Distance Between Two Line Segments	Intersection + endpoint distances	$O(1)$
Rectangle Intersection / Overlap	Interval overlap	$O(1)$
Range and Bearing Between Two Poses	Basic trigonometry	$O(1)$
Range/Bearing to Cartesian	Polar to Cartesian	$O(1)$
2D Trilateration from Three Anchors	Subtract circle equations	$O(1)$
Constant-Velocity State Prediction	Motion model	$O(1)$
1D Kalman Filter	Predict + update	$O(1)$
Innovation / Mahalanobis Gating	Normalized residual	$O(1)$
Timestamp Association	Nearest timestamp	$O(n)$ simple version

1. Closest Point on a Line Segment

Core idea: Compute the projection parameter t of point P onto segment AB .

Implementation: $t = ((P-A) \cdot (B-A)) / ||B-A||^2$. Clamp t to $[0,1]$, then $\text{closest} = A + t(B-A)$.

Remember: If $t=0$, A is closest. If $t=1$, B is closest. Values in between land inside the segment.

2. Distance Between Two Line Segments

Core idea: First test whether the segments intersect. If they do, distance is zero.

Implementation: Otherwise compute four point-to-segment distances: each endpoint of one segment to the other segment. Return the minimum.

Remember: This is simple and robust for interview settings.

3. Rectangle Intersection / Overlap

Core idea: Two axis-aligned rectangles overlap only if their x ranges overlap and their y ranges overlap.

Implementation: Reject when one rectangle is completely left, right, above, or below the other.

Remember: Touching edges are treated as no area overlap in this implementation.

4. Range and Bearing Between Two Poses

Core idea: Compute dx and dy from robot to target.

Implementation: $\text{Range} = \sqrt{dx^2 + dy^2}$. $\text{Global bearing} = \text{atan2}(dy, dx)$. $\text{Relative bearing} = \text{global bearing} - \text{robot heading}$, then wrap to $[-\pi, \pi]$.

Remember: Always wrap the final angular difference.

5. Range/Bearing to Cartesian

Core idea: Convert a polar measurement into local x/y coordinates.

Implementation: $x = r \cos(\text{theta})$, $y = r \sin(\text{theta})$.

Remember: This gives coordinates in the frame where the bearing was measured.

6. 2D Trilateration from Three Anchors

Core idea: Each range defines a circle. Subtract the first circle equation from the second and third.

Implementation: The squared x and y terms cancel, leaving two linear equations in x and y . Solve the resulting 2×2 system.

Remember: The anchors must not be collinear for a unique solution.

7. Constant-Velocity State Prediction

Core idea: Assume velocity stays constant over dt .

Implementation: $x_{next} = x + v_x \cdot dt$ and $y_{next} = y + v_y \cdot dt$. Velocity remains unchanged.

Remember: This is the standard starting point for a constant-velocity Kalman filter.

8. 1D Kalman Filter

Core idea: Prediction increases uncertainty: $P = P + Q$.

Implementation: Update uses $K = P/(P+R)$, then $x = x + K(z-x)$, and $P = (1-K)P$.

Remember: Large R trusts the measurement less. Large P gives the measurement more influence.

9. Innovation / Mahalanobis Gating

Core idea: Compare a measurement residual against its expected uncertainty.

Implementation: For one dimension, $NIS = \text{residual}^2 / S$. Accept when NIS is below the chosen threshold.

Remember: This prevents unexpectedly large measurements from entering the estimator.

10. Timestamp Association

Core idea: Find the state timestamp closest to a sensor measurement time.

Implementation: Compute absolute time difference for each timestamp and keep the smallest.

Remember: For sorted timestamps and large datasets, replace the linear scan with `lower_bound` for $O(\log n)$.